

Day 14 — Classical ML Consolidation & Model Defense

Revision summary

At senior level, the important skill is not remembering model definitions. It is being able to defend this chain under pressure:

```
Business decision
  ↓
Target definition
  ↓
Leakage-safe data
  ↓
Train / validation / test split
  ↓
Baseline
  ↓
Model comparison
  ↓
Metric aligned to business cost
  ↓
Probability calibration
  ↓
Decision threshold
  ↓
Slice / fairness / robustness checks
  ↓
Online or shadow evaluation
  ↓
Monitoring + rollback
```

The five questions you should be able to answer for almost any classical ML system are:

1. **What exactly are you predicting, and at what time is the prediction made?**
2. **Why does your split reproduce production?**
3. **Why is your primary metric aligned with the business decision?**
4. **How is the operating threshold chosen?**
5. **What would make your offline conclusion wrong?**

Priority table

Priority	Topic	Senior-level expectation
P0	Leakage	Identify target, temporal, aggregation, proxy, preprocessing, and cross-validation leakage
P0	Split strategy	Defend random vs stratified vs grouped vs temporal split
P0	Metrics	Connect metric to decision cost rather than choosing accuracy/AUC automatically
P0	Thresholding	Separate model training from business operating-point selection

Priority	Topic	Senior-level expectation
P0	Calibration	Know discrimination $\neq$ calibrated probability
P1	Trees/boosting	Explain why boosted trees often win on structured data and when they do not
P1	Feature engineering	Create features available at inference time and stable over time
P1	Experiments	Connect offline improvements to safe online validation
P1	Fairness	Evaluate relevant slices without assuming one aggregate fairness metric solves the problem
P2	Explanation	Distinguish predictive explanation from causal explanation

Part 1 — 25-question closed-book quiz

Rules

- 25 questions.
- Target: **25–30 minutes**.
- No notes.
- Answer each in 1–4 sentences.
- For model-choice questions, always give **reason + trade-off**.
- Do not scroll to the answer key until finished.

Q1

You want to predict whether an invoice will be paid more than 30 days late. A feature contains the number of collection calls made during the 45 days after invoice creation.

What is wrong?

Q2

Your fraud dataset has 0.5% positives. A model has 99.5% accuracy.

Why is this insufficient?

Q3

When is PR-AUC generally more informative than ROC-AUC?

Q4

Model A has ROC-AUC 0.91 and Model B has ROC-AUC 0.89.

Can you conclude A is the better production model?

Q5

What is the distinction between **ranking/discrimination** and **calibration**?

Q6

A classifier gives a score of 0.8 to many examples, but only roughly 55% of those examples are actually positive.

What problem does this suggest?

Q7

Why should the decision threshold often **not** default to 0.5?

Q8

A false negative costs ₹20,000 and a false positive costs ₹500.

What implication does that have for threshold selection?

Q9

Why can a random train/test split be dangerous on financial or temporal data?

Q10

You repeatedly tune hyperparameters based on performance on the test set.

What happened?

Q11

Why might logistic regression still be useful even when gradient boosting performs better?

Q12

Why can an unrestricted decision tree severely overfit?

Q13

What are the main differences between random forests and gradient boosting?

Q14

Why does lowering the learning rate in gradient boosting often require increasing the number of trees?

Q15

Give two reasons gain-based tree feature importance may be misleading.

Q16

A feature named `customer_total_defaults` is computed over the entire dataset before splitting.

Why could that be leakage even though the feature does not contain the target column directly?

Q17

When would scaling usually matter much more for logistic regression than for a decision tree?

Q18

Why can target encoding introduce leakage?

Q19

What is the difference between permutation importance and SHAP at a high level?

Q20

A SHAP explanation says income strongly decreased a customer's predicted default probability.

Can you conclude increasing income would causally reduce default risk by that amount?

Q21

Model A improves PR-AUC from [old] to [new].

What must you establish before calling the improvement meaningful?

Q22

Why are confidence intervals or paired resampling useful when comparing models?

Q23

An A/B experiment was intended to allocate users 50/50, but actual assignment is 57/43.

What should you investigate?

Q24

Why might you deploy a challenger in shadow mode before letting it make live decisions?

Q25

Give three reasons excellent offline performance might fail to translate into production impact.

STOP — complete the quiz before continuing

Score yourself only after giving a concrete answer to all 25.

Part 2 — Answer key**A1**

This is **future-information leakage**. At invoice creation time, the system does not know collection activity occurring over the following 45 days.

A2

With 0.5% positives, predicting every example as negative produces about 99.5% accuracy. Examine recall, precision, PR-AUC, costs, and the confusion matrix.

A3

PR-AUC is especially useful with **rare positive classes** when performance on positives is operationally important.

A4

No. Evaluate statistical uncertainty, practical significance, calibration, operating threshold, latency, cost, robustness, slices, and production constraints.

A5

Discrimination measures whether positives tend to rank above negatives. Calibration measures whether predicted probabilities correspond to observed frequencies.

A6

Poor probability calibration.

A7

Because the optimal operating point depends on false-positive/false-negative costs, capacity, policy, risk tolerance, and downstream actions.

A8

False negatives are much more expensive, so the cost-optimal threshold will generally favor higher recall, subject to operational constraints.

A9

Future examples can indirectly influence training, and temporal drift may make the test distribution unrealistically similar to training.

A10

You have contaminated the test set through **test-set overfitting**. It no longer provides an unbiased final estimate.

A11

It provides a strong baseline, simpler behavior, lower serving complexity, easier debugging, and often better interpretability/calibration.

A12

It can repeatedly partition until leaves represent tiny samples or noise rather than reproducible structure.

A13

Random forest builds many largely independent trees using bagging and feature subsampling. Boosting builds trees sequentially to correct prior errors.

A14

Each tree makes a smaller contribution, so more stages are usually needed to achieve comparable model capacity.

A15

Examples: correlated variables split importance among themselves; high-cardinality/continuous features can receive disproportionate split opportunities.

A16

If aggregates include events occurring after an observation's prediction time, they expose future information.

A17

Scaling affects optimization and regularization because coefficients depend on feature scale. Tree splits depend primarily on ordering, so scaling usually changes little.

A18

If the encoding for a row uses its own target or validation/test targets, target information leaks into the feature.

A19

Permutation importance measures performance degradation after disrupting a feature. SHAP attributes individual predictions using a game-theoretic feature-attribution framework.

A20

No. SHAP explains the model's predictive behavior, not a causal intervention.

A21

Check uncertainty, repeated/paired comparison, practical/business importance, relevant slices, and whether the comparison used an untouched evaluation set.

A22

Observed metric differences contain sampling noise. Paired resampling estimates uncertainty while exploiting that both models were evaluated on the same examples.

A23

Investigate **sample-ratio mismatch** before trusting experimental results.

A24

Shadow mode reveals latency, reliability, drift, integration, prediction differences, and operational failures without letting the challenger affect users.

A25

Examples: distribution shift, leakage, calibration failure, incorrect threshold, system latency, adoption failure, interventions changing behavior, poor slice performance, or delayed outcomes.

Suggested score bands

Score	Interpretation
23–25	Strong consolidation

Score	Interpretation
20–22	Interview-ready with targeted repair
16–19	Material gaps remain
≤15	Rebuild the P0 topics before progressing

Do **not** turn this into your Week 2 score until you have actually answered closed-book.

Part 3 — Model-risk review of the Week 2 PoC

Your Week 2 work covered leakage-safe features, supervised modelling, anomaly detection, metrics, calibration, model comparison, explanations, and experimentation.

I have the design requirements from the preparation track, but **not your actual repository, experiment outputs, tests, or metric values**. Therefore I will not pretend that the implementation has passed review.

As a model-risk reviewer, these are the questions I would use.

1. Target

Reviewer question: What exactly is y , when does it become observable, and what decision happens after prediction?

Pass condition:

```
observation time < prediction time < outcome measurement
```

Red flag:

```
feature window overlaps outcome window
```

You should be able to state:

“For every entity at prediction time T , the model predicts [outcome] during [future horizon]. Only information available at or before T enters the features.”

2. Data split

I would reject:

“I used `train_test_split(..., random_state=42)` because it is standard.”

I would expect:

“The split reflects production. Because observations evolve over time, older periods train the model, a later interval supports model selection, and the newest untouched interval estimates final generalization.”

If multiple rows belong to the same customer/vendor/account, also test whether an **entity-group constraint** is needed.

3. Leakage controls

Inspect:

- feature timestamps
- target-derived aggregates
- target encoding
- normalization/scaling fit
- imputation fit
- feature selection
- PCA
- resampling
- anomaly detector training
- duplicate entities
- post-outcome events

Correct pattern:

```
Raw data
  ↓
Split first
  ↓
Fit transformations on training only
  ↓
Transform validation/test
```

4. Baseline

A sophisticated model without a meaningful baseline is weak evidence.

Expected comparisons might include:

```
naive/business rule
  ↓
logistic regression
  ↓
tree / random forest
  ↓
gradient boosting
```

The question is not:

| Which model has the largest metric?

It is:

| Does additional complexity produce enough reliable value to justify operational cost and risk?

5. Evaluation

I want to see clearly separated:

```
Model quality
├── discrimination
├── calibration
└── slice robustness

Business policy
└── threshold

Operational system
├── latency
├── cost
└── reliability
```

A common mistake is compressing all three into one AUC value.

6. Threshold

The threshold should be selected using validation data.

Then it should be **frozen** before reporting final test results.

Defensible answer:

```
"I selected the threshold based on the asymmetric cost of false positives and false negatives, subject to operational review capacity. I did not tune it against the final test set."
```

7. Calibration

For models producing actionable probabilities, inspect:

- reliability curve
- Brier score
- calibration by important segment
- Platt scaling or isotonic calibration where justified

And importantly:

```
train model
  ↓
calibration data
  ↓
fit calibrator
  ↓
untouched evaluation
```

Do not calibrate using the final test labels.

8. Unsupervised anomaly detector

Isolation Forest or another anomaly detector should not automatically be compared with supervised classification as though they solve exactly the same problem.

Ask:

| Does “anomalous” actually mean “risky”?

Evaluation under limited labels should consider:

- precision@review-capacity
 - known-positive enrichment
 - expert review
 - temporal backtesting
 - stability
-

9. Explainability

Pass:

| “SHAP helps us understand how the trained model used features.”

Fail:

| “SHAP proves these variables cause the outcome.”

Also inspect explanation stability and correlated features.

10. Reproducibility

Minimum evidence:

```
source revision
dataset/version
feature definitions
split boundaries
random seeds where applicable
library versions
hyperparameters
model artifact
threshold
calibrator
evaluation report
```

11. Tests

A Week 2 PoC should have at least conceptual coverage for:

- feature schema validation
- temporal boundary tests

- leakage regression test
- missing-value behavior
- unseen category behavior
- deterministic preprocessing
- threshold boundary
- metric correctness
- model serialization/deserialization
- inference schema

A notebook that “runs successfully” is not sufficient testing.

Model-risk verdict

Without seeing the actual implementation:

Status: NOT YET APPROVABLE — evidence incomplete.

That is not a negative judgement about your model. It is the correct risk-review conclusion when the implementation evidence and experimental results have not been inspected.

Part 4 — Five-minute model defense

You get exactly five minutes.

Use this structure.

0:00–1:00 — Target

“The model predicts [target] for [unit] over [prediction horizon]. Predictions are generated at [decision point], and all features are restricted to information available by that time.”

1:00–2:00 — Split

“I used [temporal/grouped/stratified] validation because [reason tied to production]. The final test period remained untouched during model and threshold selection.”

2:00–3:00 — Metric

“The primary metric is [metric] because [business consequence]. I also track [secondary metrics] because no single aggregate metric captures discrimination, calibration, and operating performance.”

3:00–4:00 — Threshold

“The model outputs a score/probability. The decision threshold is a separate business-policy choice selected using [cost/capacity/constraint] on validation data and then frozen.”

4:00–5:00 — Limitations

Name at least four:

```
distribution shift
limited labels
segment instability
proxy variables
calibration drift
unobserved confounding
delayed outcomes
reviewer feedback loops
```

If you spend four minutes discussing algorithms and twenty seconds discussing limitations, I would mark the answer down.

Part 5 — Model comparison without notes

Prompt

You trained:

- logistic regression
- random forest
- gradient-boosted trees

Gradient boosting has the strongest validation discrimination.

You have **90 seconds**:

| Which model do you deploy?

Do not answer merely:

| “Gradient boosting because it has the best metric.”

A strong response should cover:

```
performance difference
+ uncertainty
+ calibration
+ threshold performance
+ segment stability
+ temporal robustness
+ explainability
+ latency/cost
+ reproducibility
+ business materiality
```

Reference answer

“I would not deploy solely from the ranking metric. I would first determine whether the boosted model's advantage over logistic regression and random forest is statistically and practically meaningful on an untouched temporal evaluation set. I would compare calibration and performance at the actual business threshold, including important slices. If boosting retains a meaningful advantage without unacceptable latency, stability, or explainability costs, I would deploy it. Otherwise, logistic regression may be preferable because operational simplicity can outweigh a small offline gain.”

Part 6 — 30-minute applied ML case interview

Case

A company processes a large volume of supplier invoices.

Finance operations wants an ML system that identifies invoices likely to become **more than 30 days overdue**, so reviewers can intervene before payment problems escalate.

The historical dataset contains:

- supplier ID
- supplier country
- invoice amount
- purchase category
- contract type
- invoice date
- historical payment behavior
- current disputed status
- business unit
- requester
- approval-chain characteristics
- payment date for historical invoices
- collection actions
- textual invoice descriptions

Only a limited number of invoices can be manually reviewed daily.

Your job

Design the ML approach.

Minute 0–5 — Frame the problem

I expect you to clarify:

- unit of prediction
- prediction time
- outcome horizon

- intervention
- review capacity
- false-positive cost
- false-negative cost

Interviewer challenge:

| Why are you building ML instead of a rule?

Minute 5–10 — Data and leakage

Identify at least five leakage risks.

Possible traps:

```
final payment date
post-invoice disputes
future collection calls
aggregate supplier statistics using future invoices
approval status finalized after prediction time
```

Interviewer challenge:

| Supplier history is legitimate. Why can supplier history still leak?

Expected concept:

point-in-time correctness.

Minute 10–15 — Split and features

Defend a split.

Strong default:

```
Train      Validation      Test
older -----> newer
```

Potential complications:

- seasonality
- repeated suppliers
- newly onboarded suppliers
- policy changes

Interviewer challenge:

| Random splitting gives you more balanced datasets. Why not use it?

Minute 15–20 — Models

Choose candidates.

A solid answer:

1. simple business-rule baseline
2. logistic regression
3. gradient-boosted tree
4. optionally random forest as another comparison

Potential text features should only be added if they provide material incremental value and preserve production correctness.

Interviewer challenge:

| Why not use a neural network?

Good response:

| “I wouldn’t begin with one purely because it is more expressive. For moderate structured tabular data, boosted trees provide an excellent accuracy/engineering-cost baseline. I would introduce neural approaches only if data modality, scale, or measured incremental value justified them.”

Minute 20–25 — Evaluation and threshold

You must distinguish:

Ranking

PR-AUC / ROC-AUC.

Probability quality

Calibration / Brier score.

Operational decision

Precision, recall, cost, and review-volume at the chosen threshold.

An especially strong answer recognizes:

| If only 500 invoices/day can be investigated, precision or recall **at review capacity** may matter more than an abstract global threshold.

Minute 25–28 — Fairness and robustness

Evaluate slices such as:

- supplier geography
- supplier size
- new vs established supplier
- business unit

- purchasing category

But do not casually promise “equal performance across every group.”

Ask whether group differences represent:

- data quality
- base-rate differences
- legitimate predictive structure
- problematic proxies
- historical process bias

Minute 28–30 — Deployment

Good approach:

```
Offline validation
  ↓
Shadow
  ↓
Compare production distributions
  ↓
Reviewer-only recommendations
  ↓
Measure intervention effects
  ↓
Canary / phased rollout
```

Final interviewer question

Your model has excellent offline recall. Three months after launch, overdue invoices have not decreased. Explain.

I expect hypotheses including:

- reviewers ignore recommendations
- intervention is ineffective
- prediction arrives too late
- capacity is constrained
- distribution shifted
- threshold is wrong
- model finds cases humans cannot influence
- measurement window is too short
- feedback loops changed the population

The key senior insight:

Predictive quality and intervention effectiveness are different questions.

Part 7 — 20-minute coding mock

Problem

Implement a function that chooses a classification threshold based on asymmetric business cost.

```
def choose_threshold(
    y_true: list[int],
    y_prob: list[float],
    false_positive_cost: float,
    false_negative_cost: float,
) -> tuple[float, float]:
    ...
```

Return:

```
(best_threshold, minimum_total_cost)
```

For threshold t :

```
prediction = 1 if probability >= t else 0
```

Total cost:

```
FP * false_positive_cost
+
FN * false_negative_cost
```

Requirements

- Do not use sklearn.
- Probabilities are between 0 and 1.
- Include threshold candidates that allow predicting all negative and all positive.
- State tie-breaking behavior.
- Discuss complexity.

What I am evaluating

Not Python syntax.

I care about whether you understand:

```
probability
    ≠
decision

model
    ≠
business policy
```

STOP — code before reading the reference solution

Reference solution

```
def choose_threshold(
    y_true: list[int],
    y_prob: list[float],
    false_positive_cost: float,
    false_negative_cost: float,
) -> tuple[float, float]:

    if len(y_true) != len(y_prob):
        raise ValueError("y_true and y_prob must have the same length")

    if not y_true:
        raise ValueError("inputs cannot be empty")

    if any(y not in (0, 1) for y in y_true):
        raise ValueError("y_true must contain only 0 or 1")

    if any(p < 0 or p > 1 for p in y_prob):
        raise ValueError("probabilities must be between 0 and 1")

    thresholds = sorted(set(y_prob))
    thresholds = [0.0] + thresholds + [1.0 + 1e-12]

    best_threshold = None
    best_cost = float("inf")

    for threshold in thresholds:
        fp = 0
        fn = 0

        for y, p in zip(y_true, y_prob):
            pred = int(p >= threshold)

            if pred == 1 and y == 0:
                fp += 1
            elif pred == 0 and y == 1:
                fn += 1

        cost = (
            fp * false_positive_cost
            + fn * false_negative_cost
        )

        if cost < best_cost:
            best_cost = cost
            best_threshold = threshold
```

```
return best_threshold, best_cost
```

Complexity of this straightforward implementation:

k unique thresholds
n observations

Time: $O(k \times n)$
Worst case: $O(n^2)$

Space: $O(k)$

A stronger candidate may propose sorting probabilities once and updating FP/FN incrementally, reducing the main computation toward:

$O(n \log n)$

Senior follow-up

Would you select this threshold directly on the test set?

Correct answer:

No.

Select using validation data and report final performance once on the untouched test set.

Part 8 — High-risk misunderstandings

I cannot truthfully call these **your** misunderstandings until I have your quiz/case answers.

These are the ten misconceptions I would actively test because they commonly separate mid-level from senior applied-ML answers.

Misunderstanding	Correct mental model
Best AUC = best production model	Production choice includes uncertainty, calibration, costs and constraints
0.5 is the natural threshold	Threshold is an operating-policy decision
High ROC-AUC means good probabilities	Ranking and calibration are different
Leakage means literally including y	Future information and target-derived transformations also leak
Cross-validation eliminates leakage	Incorrect preprocessing can leak across every fold
SHAP explains causality	SHAP explains model attribution
Accuracy is enough	Depends on class balance and costs
Random split is neutral	Temporal/entity structure can make it unrealistic

Misunderstanding	Correct mental model
Statistical significance means business value	Practical significance must also matter
Offline uplift proves business impact	Deployment/intervention needs separate validation

Part 9 — Weak-area recovery register

After scoring the quiz, update this table.

Quiz misses	Weak area	Recovery exercise
1, 9, 16, 18	Leakage	Take 10 candidate features and assign an explicit availability timestamp
2, 3, 4	Metrics	Explain ROC-AUC vs PR-AUC vs operating metrics in 90 seconds
5, 6	Calibration	Draw a reliability curve and explain over/under-confidence
7, 8	Thresholding	Solve three asymmetric-cost threshold examples manually
10, 21, 22	Evaluation	Design train/validation/test + paired comparison
11–15	Trees/boosting	Compare logistic, RF and boosting without model definitions
19, 20	Explainability	Explain predictive vs causal explanations
23–25	Experimentation	Diagnose SRM, shadow rollout and offline/online mismatch

Recovery rule:

Miss once → explain it
Miss twice → implement it
Miss again → design a production failure around it

Part 10 — Ten flashcards

Use these as the initial deck; replace cards covering concepts you got right with concepts you actually missed.

- Q:** Does ROC-AUC measure probability calibration? **A:** No. It primarily measures ranking/discrimination.
- Q:** Where should an operating threshold be selected? **A:** On validation data using business costs/capacity, then frozen before final test evaluation.
- Q:** What is point-in-time correctness? **A:** Every feature must contain only information available at the prediction timestamp.

4. **Q:** Why can aggregate features leak? **A:** Their calculation may incorporate future observations.
5. **Q:** PR-AUC is most useful when? **A:** Positives are rare and retrieving them effectively matters.
6. **Q:** Does SHAP establish causality? **A:** No. It attributes a model prediction.
7. **Q:** Why keep a logistic baseline? **A:** It provides a simple, interpretable, inexpensive reference for incremental model value.
8. **Q:** Statistical vs practical significance? **A:** Statistical asks whether an effect is likely real; practical asks whether it is large enough to matter.
9. **Q:** What is sample-ratio mismatch? **A:** Observed experimental allocation materially differs from intended randomization.
10. **Q:** Why shadow a model? **A:** Observe real production behavior without letting its predictions affect decisions.
-

Part 11 — Executive explanation

We built the model to identify cases that are more likely to experience the target outcome early enough for the business to intervene.

The model does not make the final business decision. It produces a risk score using information available at prediction time. We validate it on later data so that our evaluation better reflects how the system will encounter future cases in production.

We compare the model against simpler baselines and evaluate not only how well it ranks risk, but also whether its probabilities are reliable and how it performs at the operating threshold the business can actually support.

The threshold is chosen separately from model training because false positives, false negatives, and review capacity have different business consequences.

The main limitations are that customer and operational behaviour can change, probabilities can drift, some segments may perform differently, and a good prediction does not guarantee that the downstream intervention will improve the business outcome. For those reasons, the model should be monitored and rolled out progressively rather than treated as a one-time prediction exercise.

Part 12 — One-page model decision memo

Model Decision Memo

Decision

Evaluate [selected model] as the preferred candidate for [business decision], subject to final confirmation on the untouched evaluation set, slice analysis, calibration, operational constraints, and deployment validation.

Target

The model predicts [target] for [prediction entity] over [prediction horizon].

Predictions are generated at [prediction time]. Features must contain only information available at or before this point.

Data and Validation

The evaluation uses [temporal/grouped/other] splitting because it most closely reproduces the expected production setting.

Preprocessing, feature transformations, sampling, feature selection, and calibration are fitted without using final test information.

The final evaluation dataset remains untouched during model selection and threshold tuning.

Model Comparison

Candidates include:

- simple/business baseline
- logistic regression
- [tree/random forest]
- gradient-boosted trees

The selected model should not be chosen from one metric alone. Selection considers predictive performance, uncertainty, calibration, important segment performance, stability, explainability, serving complexity, latency, and operational cost.

No numerical model advantage is asserted in this memo until validated results are available.

Evaluation and Operating Point

Primary metric: [metric]

Reason: [relationship to business objective]

Secondary checks include [precision/recall/PR-AUC/ROC-AUC/calibration/Brier score/slice metrics].

The operating threshold is selected independently using [false-positive cost, false-negative cost, review capacity or policy constraint] on validation data and frozen before final testing.

Explainability and Risk

Feature-attribution methods may be used to understand model behaviour but are not interpreted as causal effects.

Important slices should be evaluated for data-quality problems, instability, proxy effects, and material performance differences.

Limitations

Material limitations include:

- future distribution shift
- calibration drift
- incomplete or delayed labels
- performance variation across segments
- sensitivity to feature-data quality
- potential feedback effects after deployment
- uncertainty that improved prediction will produce improved business outcomes

Deployment Recommendation

Use progressive validation:

Offline evaluation → shadow mode → controlled/champion-challenger comparison → limited rollout → monitored expansion.

Rollback criteria should cover model quality, calibration, data quality, system reliability, latency, and business guardrails.

Final Approval Condition

Production approval requires reproducible evidence for the data version, feature definitions, split, model artifact, calibration method, threshold, evaluation results, relevant tests, monitoring plan, and ownership.

Week 2 scorecard

Do not fill the achieved column from memory or optimism. Score it from today's actual performance.

Area	Weight	Evidence	Your score
Lifecycle + leakage	15	Q1/Q9/Q16/Q18 + case	/15
Metrics + imbalance	10	Q2–Q4	/10
Calibration + threshold	15	Q5–Q8 + coding	/15
Regression/classification fundamentals	10	verbal defense	/10
Trees + boosting	15	Q11–Q15 + comparison	/15
Feature engineering	10	case + PoC review	/10
Explainability/fairness	10	Q19/Q20 + case	/10
Experimentation/deployment	10	Q21–Q25	/10
Executive communication	5	five-minute defense	/5
Total	100		/100

Rating

90–100	Strong
80–89	Ready, targeted cleanup

70–79	Uneven; repair P0 gaps
<70	Consolidate before increasing difficulty

PoC quality gate

Separately score your implementation:

Dimension	Pass condition
Correctness	Point-in-time feature correctness established
Reproducibility	Environment, split, seed/config and model artifact reproducible
Tests	Data, feature, inference and leakage-sensitive tests present
Evaluation	Untouched final evaluation + suitable metrics
Calibration	Checked where probabilities drive decisions
Threshold	Explicit business justification
Comparison	Baseline and alternatives evaluated fairly
Explanation	Predictive explanation presented without causal claims
Risk	Limitations and relevant slices documented
Deployment	Monitoring and rollout strategy stated

Next-week priorities

Your priority order should be determined by today's misses, but unless the score says otherwise:

- 1. Make leakage reasoning automatic.** Every feature should trigger the question: *Could I know this at prediction time?*
- 2. Stop collapsing evaluation into one metric.**

Ranking → calibration → threshold → business outcome

Treat those as separate layers.

3. Defend model complexity. Be able to explain why boosting earns its operational complexity relative to logistic regression.

4. Strengthen experimental reasoning. Offline superiority does not establish causal business impact.

5. Practise the five-minute defense repeatedly. A senior answer should communicate the target, validation design, metric, operating point, and limitations before discussing algorithm trivia.

The highest-value next step is to answer the **25 quiz questions closed-book**, give yourself the raw /25, and then do the 30-minute case without reading the supplied checkpoints. That will give you a real Day 14 score rather than an invented one.

Day 14 DSA Track — Sorting

1. Recognition signals

Think **sorting** when the problem contains signals such as:

- “Arrange by increasing/decreasing...”
- “Closest pairs”, “overlapping intervals”, “meeting schedules”
- “Group equal values together”
- “Find duplicates after ordering”
- “Sort using multiple fields”
- “Rearrange according to a condition”
- “Partition into smaller/equal/larger groups”
- The problem becomes much easier once neighboring values have meaningful order.

Typical transformation:

```
Unordered data
  ↓
Sort / partition
  ↓
Local relationships become useful
  ↓
Single linear scan
```

A common complexity pattern is:

```
sorting:  $O(n \log n)$ 
scan:     $O(n)$ 
-----
total:    $O(n \log n)$ 
```

2. Core sorting concepts

Comparison sorts

A comparison sort determines ordering using comparisons such as:

```
a < b
a > b
```

Examples:

Algorithm	Average	Worst	Stable?	Extra space
Bubble Sort	$O(n^2)$	$O(n^2)$	Yes	$O(1)$
Insertion Sort	$O(n^2)$	$O(n^2)$	Yes	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	No	$O(1)$

Algorithm	Average	Worst	Stable?	Extra space
Merge Sort	$O(n \log n)$	$O(n \log n)$	Yes	$O(n)$
Quick Sort	$O(n \log n)$	$O(n^2)$	Usually no	$O(\log n)$ avg stack
Heap Sort	$O(n \log n)$	$O(n \log n)$	No	$O(1)$

General comparison-based sorting has a lower bound of:

$\Omega(n \log n)$

for arbitrary values.

3. Stability

A sorting algorithm is **stable** when items with equal sorting keys retain their original relative order.

Suppose:

(name, score)

A → 90

B → 80

C → 90

Sorting by score descending with a stable sort gives:

A → 90

C → 90

B → 80

A remains before C.

This matters when sorting records repeatedly.

Example:

```
employees = [
    ("Alice", "Engineering"),
    ("Bob", "Sales"),
    ("Charlie", "Engineering"),
]
```

Python's built-in sorting is stable.

4. Custom keys

In Python, prefer `key=` rather than writing manual comparison logic.

```
users = [  
    {"name": "Alice", "age": 35},  
    {"name": "Bob", "age": 28},  
    {"name": "Charlie", "age": 31},  
]  
  
users.sort(key=lambda x: x["age"])
```

Multiple keys:

```
users.sort(  
    key=lambda x: (x["age"], x["name"])  
)
```

Descending:

```
users.sort(  
    key=lambda x: x["age"],  
    reverse=True  
)
```

A common interview pattern:

```
intervals.sort(key=lambda x: x[0])
```

Then process intervals from left to right.

5. Partitioning

Partitioning does **not necessarily fully sort** the collection.

It rearranges values according to a condition.

For example:

```
Original:  
5 2 8 1 7 3  
  
Partition around 5:  
  
2 1 3 | 5 | 8 7  
< 5    > 5
```

Quick Sort relies on partitioning.

Another important pattern is **three-way partitioning**:

```
< pivot | == pivot | > pivot
```

This leads directly to today's problem.

Medium Problem — Sort Colors

Problem

Given an array containing only:

```
0, 1, 2
```

sort it in-place so that:

```
0s come first  
1s come second  
2s come last
```

Example:

```
nums = [2, 0, 2, 1, 1, 0]
```

Result:

```
[0, 0, 1, 1, 2, 2]
```

Constraint:

Try to solve it:

- in one pass
- using $O(1)$ extra space

6. Recognition signals

Important clues:

- only **three possible values**
- sorting is required
- in-place requested
- one-pass requested
- values naturally form three partitions

That should suggest:

| **Dutch National Flag / three-way partitioning**

We want:

```
0-region | unknown | 2-region
```

and gradually eliminate the unknown region.

7. Brute-force reasoning

The simplest solution is:

```
nums.sort()
```

Code

```
def sort_colors(nums: list[int]) -> None:
    nums.sort()
```

Python's sort gives:

```
Time:  O(n log n)
Space: implementation-dependent
```

It works, but it ignores the stronger fact:

| There are only three distinct values.

8. Better reasoning — counting

Count the number of:

```
0s
1s
2s
```

Then overwrite the array.

Example:

```
[2,0,2,1,1,0]

counts:
0 → 2
1 → 2
2 → 2

rewrite:
[0,0,1,1,2,2]
```

Python

```
def sort_colors(nums: list[int]) -> None:
    zeros = ones = twos = 0
```

```

for value in nums:
    if value == 0:
        zeros += 1
    elif value == 1:
        ones += 1
    else:
        twos += 1

index = 0

for _ in range(zeros):
    nums[index] = 0
    index += 1

for _ in range(ones):
    nums[index] = 1
    index += 1

for _ in range(twos):
    nums[index] = 2
    index += 1

```

Complexity:

```

Time: O(n)
Space: O(1)

```

But this requires effectively two passes.

The requested optimal solution can do it in **one traversal**.

9. Optimized reasoning — Dutch National Flag

Maintain three pointers:

```

low
mid
high

```

Their meaning:

```

[ 0 region ][ 1 region ][ unknown ][ 2 region ]
      ↑       ↑       ↑
      low    mid    high

```

More precisely:

```

0 ... low-1    → all 0
low ... mid-1  → all 1

```

```
mid ... high      → unknown
high+1 ... n-1    → all 2
```

We examine:

```
nums[mid]
```

There are only three cases.

Case 1: `nums[mid] == 0`

Zero belongs on the left.

Swap:

```
nums[low] ↔ nums[mid]
```

Then:

```
low += 1
mid += 1
```

Case 2: `nums[mid] == 1`

One is already in the correct middle region.

Just:

```
mid += 1
```

Case 3: `nums[mid] == 2`

Two belongs on the right.

Swap:

```
nums[mid] ↔ nums[high]
```

Then:

```
high -= 1
```

But do **not** increment mid.

Why?

Because the value brought from high has not been inspected yet.

This is the critical part of the algorithm.

10. Pseudocode

```
low  = 0
mid  = 0
high = n - 1

while mid <= high:

    if nums[mid] == 0:
        swap nums[mid], nums[low]
        low++
        mid++

    else if nums[mid] == 1:
        mid++

    else:
        swap nums[mid], nums[high]
        high--
```

11. Python solution

```
def sort_colors(nums: list[int]) -> None:
    low = 0
    mid = 0
    high = len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1

        elif nums[mid] == 1:
            mid += 1

        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

12. Walkthrough

Input:

```
[2, 0, 2, 1, 1, 0]
```

Initially:


```
low = 0
mid = 0
high = 5

[2, 0, 2, 1, 1, 0]
  ^
mid
```

nums[mid] == 2

Swap indices 0 and 5:

```
[0, 0, 2, 1, 1, 2]

low = 0
mid = 0
high = 4
```

Notice:

```
mid did NOT move
```

Now nums[mid] == 0.

Swap with low:

```
[0, 0, 2, 1, 1, 2]

low = 1
mid = 1
```

Again zero:

```
[0, 0, 2, 1, 1, 2]

low = 2
mid = 2
```

Now 2.

Swap with index 4:

```
[0, 0, 1, 1, 2, 2]

high = 3
mid = 2
```

Now 1:

```
mid = 3
```

Again 1:

```
mid = 4
```

Now:

```
mid > high
```

Done.

13. Correctness invariant

At every step:

```
nums[0 : low]      are all 0
nums[low : mid]    are all 1
nums[mid : high+1] are unknown
nums[high+1 :]    are all 2
```

Each iteration reduces the unknown region.

Eventually:

```
mid > high
```

so there are no unknown elements remaining.

Therefore the array is correctly partitioned:

```
0s | 1s | 2s
```

14. Edge cases

Empty array

```
[]
```

Works automatically.

One item

```
[1]
```

Works.

All zeros

```
[0, 0, 0]
```

low and mid move together.

All twos

```
[2, 2, 2]
```

Only high moves.

Already sorted

```
[0, 0, 1, 1, 2, 2]
```

Still $O(n)$.

Reverse order

```
[2, 2, 1, 1, 0, 0]
```

Still $O(n)$.

Critical bug

Do not write:

```
else:
    nums[mid], nums[high] = nums[high], nums[mid]
    high -= 1
    mid += 1 # wrong
```

The newly swapped value at mid is still unknown.

15. Complexity

Built-in sort

```
Time:  $O(n \log n)$ 
```

Counting approach

```
Time:  $O(n)$ 
Space:  $O(1)$ 
Passes:  $\sim 2$ 
```

Dutch National Flag

Time: $O(n)$
Space: $O(1)$
Passes: 1

Each element is processed only a constant number of times.

16. Interview explanation

A concise answer:

“Because the array contains only 0, 1, and 2, I don't need a general $O(n \log n)$ comparison sort. I can treat this as a three-way partitioning problem. I maintain low, mid, and high: everything before low is 0, everything between low and mid is 1, and everything after high is 2. If `nums[mid]` is 0, I move it left; if it is 1, I advance; if it is 2, I move it right. After swapping with high, I do not advance mid because the incoming value hasn't been classified. This gives $O(n)$ time and $O(1)$ space.”

Day 14 pattern takeaway

The major sorting patterns to recognize are:

Need full ordering
→ built-in sort / $O(n \log n)$

Need sort records
→ custom key

Need preserve ties
→ stable sorting

Sort + scan
→ intervals / pairs / grouping

Only a few categories
→ counting / partitioning

Need kth element, not full ordering
→ partition / heap

Three categories
→ Dutch National Flag

For this problem, the key leap is:

Don't perform a general sort when the value domain itself gives you a linear-time partitioning solution.